GRASP: Introducción

- Dado un problema de optimización combinatoria, GRASP es una técnica iterativa con múltiples puntos de inicio; cada iteración proporciona una solución al problema que se esté resolviendo.

- Cada iteración de GRASP consiste de dos fases:

  1. Una fase de construcción, en la que se genera una solución factible para el problema que se esté resolviendo.
  2. Una fase de búsqueda local, en la que se busca un óptimo local en la vecindad de la solución construida en la primera fase.

- En todo momento se mantiene la mejor solución obtenida, la cual será el resultado final de GRASP.

GRASP: Esquema General (Minimización)

Algoritmo 1: Esquema básico de GRASP (minimización)
Entrada: Una instancia de un problema de optimización $q = (p, S_p, c|_{S_p})$.
Salida: Una solución factible $s^* \in S_p$

GRASP(q)
(1) $c|_{S_p}(s^*) \leftarrow +\infty$;
(2) **while** (Criterio de terminación no satisfecho)
(3) $s_0 \leftarrow$ CONSTRUIR-SOLUCIÓN-INICIAL();
(4) $s^L \leftarrow$ BÚSQUEDA-LOCAL(q, s_0);
(5) **if** $\left(c|_{S_p}(s^L) < c|_{S_p}(s^*)\right)$
(6) ACTUALIZAR-SOLUCIÓN(s^*, s^L)
(7) **return** s^*

GRASP: Esquema General (Maximización)

Algoritmo 2: Esquema básico de GRASP (maximización)
Entrada: Una instancia de un problema de optimización $q = (p, S_p, c|_{S_p})$.
Salida: Una solución factible $s^* \in S_p$

GRASP(q)
(1) $c|_{S_p}(s^*) \leftarrow -\infty$;
(2) **while** (Criterio de terminación no satisfecho)
(3) $s_0 \leftarrow$ CONSTRUIR-SOLUCIÓN-INICIAL(α);
(4) $s^L \leftarrow$ BÚSQUEDA-LOCAL(q, s_0);
(5) **if** $\left(c|_{S_p}(s^L) > c|_{S_p}(s^*)\right)$
(6) ACTUALIZAR-SOLUCIÓN(s^*, s^L)
(7) **return** s^*

GRASP: Esquema general

- Las líneas (2) - (6) corresponden con las iteraciones GRASP, las cuales terminan cuando se satisface un criterio de terminación.

- El criterio de terminación puede ser:
  - Una máxima cantidad de tiempo de CPU.
  - Un máximo número de iteraciones.
  - Un máximo número de iteraciones consecutivas sin modificar la solución actual.
  - La generación de una solución s tal que $c|_{S_p}(s)$ sea mejor que un valor predefinido.
  - La combinación de algunas de estas condiciones.

GRASP: Esquema general

- La línea (3) corresponde con la fase de construcción, en la que se genera una solución factible para el problema que se esté resolviendo.

- La línea (4) corresponde con la fase de búsqueda local, en la que se busca un óptimo local en la vecindad de la solución obtenida en la fase de construcción.

- La actualización de la mejor solución obtenida hasta el momento se realiza en las líneas (5) y (6).

GRASP: Fase de Construcción

Algoritmo 3: Fase de construcción de GRASP
Entrada: Un valor $\alpha \in [0, 1]$ que controla el comportamiento aleatorio / voraz del algoritmo
Salida: Una solución factible $x \in S_p$

CONSTRUIR-SOLUCIÓN-INICIAL(α)
(1) $x \leftarrow \varnothing$;
(2) INICIALIZAR-LISTA-CANDIDATOS(C);
(3) **while** $(C \neq \varnothing)$
(4) $\quad RCL \leftarrow$ CONSTRUIR-RCL(C, α);
(5) $\quad s \leftarrow$ SELECCIONAR-ALEATORIAMENTE(RCL);
(6) $\quad x \leftarrow x \cup \{s\}$;
(7) $\quad$ ACTUALIZAR-LISTA-CANDIDATOS(C);
(8) **return** x

GRASP: Fase de Construcción

Algoritmo 4: Construcción de la RCL de GRASP (Minimización)
Entrada: Una lista de candidatos C, un valor $\alpha \in [0, 1]$ que controla el comportamiento aleatorio / voraz del algoritmo
Salida: Una lista restringida de candidatos RCL

$\textsc{Construir-RCL}(C, \alpha)$

(1) $c_{min} \leftarrow \text{mín}\{g(t) : t \in C\}$;
(2) $c_{max} \leftarrow \text{máx}\{g(t) : t \in C\}$;
(3) $RCL \leftarrow \{s \in C : g(s) \leq s_{min} + \alpha(s_{max} - s_{min})\}$;
(4) **return** RCL

GRASP: Fase de Construcción

- En la fase de construcción, se genera de manera iterativa una solución factible, agregando un elemento en cada iteración.

- En cada iteración de la fase de construcción, la selección del próximo elemento a agregar a la solución se determina ordenando los elementos en una lista de candidatos C con respecto a una función *greedy* $g : C \to \mathbb{R}$.

- La función g cuantifica el beneficio de seleccionar cada elemento.

- El componente adaptativo de GRASP está caracterizado por la actualización, en cada iteración de la fase de construcción, de los beneficios asociados con cada elemento, para reflejar los cambios producidos por la selección del elemento previo.

- El componente probabilístico de GRASP está caracterizado por la selección aleatoria del próximo elemento a agregar a la solución entre los mejores candidatos en la lista de candidatos, pero no necesariamente el mejor.

GRASP: Fase de Construcción

- La lista de los mejores candidatos se denomina *Restricted Candidate List* (RCL).

- La técnica de selección aleatoria del próximo elemento a agregar a la solución permite la generación de diferentes soluciones.

- El parámetro α controla el comportamiento aleatorio / voraz del algoritmo.
  - $\alpha = 0$ corresponde con un procedimiento de construcción voraz.
  - $\alpha = 1$ corresponde con un procedimiento de construcción aleatorio.

- La fase de construcción no garantiza la generación de soluciones que sean óptimos locales con respecto a una vecindad. Por esta razón, se realiza una búsqueda local para intentar mejorar cada solución construida.

GRASP: Fase de Búsqueda Local

Algoritmo 5: Algoritmo de Búsqueda Local (Minimización)
Entrada: Una instancia de un problema de optimización $q = (p, S_p, c|_{S_p})$ y una solución factible $s_0 \in S_p$
Salida: Un óptimo local s^L en la vecindad de s_0

BÚSQUEDA-LOCAL(q, s_0)

(1) $s^L \leftarrow s_0$;
(2) **repeat**
(3) Seleccionar $s \in N(s^L)$ tal que $c|_{S_p}(s) < c|_{S_p}(s^L)$;
(4) $s^L \leftarrow s$;
(5) **until** $\langle \forall s \in N(s^L) :: c|_{S_p}(s) > c|_{S_p}(s^L) \rangle$;
(6) **return** s^L

GRASP: Fase de Búsqueda Local

- La fase de búsqueda local es una técnica iterativa que consiste en reemplazar de manera sucesiva la solución actual por una mejor solución en la vecindad de la solución actual.

- Esta fase finaliza cuando no sea posible mejorar la solución en la vecindad.

- La mejor solución encontrada en esta fase corresponde con la mejor solución encontrada en la iteración GRASP correspondiente.

GRASP: Ventajas

- Facilidad de implementación con respecto a otras metaheurísticas.
- Requiere pocos parámetros de entonación, por lo que el desarrollo puede enfocarse en la implementación de estructuras de datos eficientes para garantizar rapidez en las iteraciones.
- Ha sido utilizado en la resolución de diversidad de problemas de optimización combinatoria, con énfasis en la eficiencia con respecto a tiempo de ejecución.

GRASP: Extensiones

- Utilización de memoria a largo plazo: el esquema básico de GRASP no utiliza la información de iteraciones previas para orientar el proceso de búsqueda de soluciones.

- GRASP reactivo: el esquema básico de GRASP utiliza valores fijos para el parámetro α. GRASP reactivo ajusta el valor de α en el transcurso de las iteraciones GRASP.

- GRASP paralelo: una alternativa consiste en descomponer el espacio de búsqueda de soluciones en regiones y aplicar GRASP a cada una de ellas en paralelo. Otro enfoque consiste en particionar las iteraciones y asignar cada partición a un procesador.